



**Sarcasm Detection System
Using Custom Naive Bayes Implementations**

**Student Name: Shivaani Raju
Student No: R00277239**

**Master of Science in Data Science and Analytics, Department of
Mathematics, 2025/26**

Declaration of Authorship

I, **Shivaani Raju**, declare that the work submitted is my own.

- I declare that I have acknowledged all main sources of help
- I declare that where I have used ChatGPT, generative (Gen) AI software, or other similar software in this assignment I have clearly stated where and why I have used them
- I declare that where I have used a language translation software in the completion of this assignment I have clearly stated where and why I have used it.
- I declare that I have used grammar correction software (Grammarly) to fix grammatical mistakes
- I declare that I have not obtained unfair assistance via use of a third party.
- I confirm that I have read and understood the policy and procedures concerning academic honesty, plagiarism, and infringements.
- I understand that where breaches of this declaration are detected, these will be reviewed under MTU (Cork) policy and procedures concerning academic honesty, plagiarism, and infringements, as well as any other University regulations and policies which may apply to the case. I also understand that any breach of academic honesty is a serious issue and may incur penalties.
- Examination/assessment material may, at the discretion of the internal examiner, be submitted to the University's plagiarism detection solution.
- I understand that I may be asked to present for an oral examination as part of this assessment.
- Where I have consulted the published work of others, this is always clearly attributed using appropriate citations and references.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this work is entirely my own work.

Signed: Shivaani Raju

Date : 27/03/2026

TABLE OF CONTENTS

1. Dataset Overview and Individualisation	5
2. Task 1: Tokenisation and Preprocessing.....	6
2.1) Basic Cleaning.....	6
2.2) Negation Handling.....	6
2.3) Stopword Removal	6
2.4) N-gram Generation.....	6
2.5) OOV Handling.....	6
3. Task 2: Manual Feature Representation.....	7
3.1) Count Vectoriser with Prime-Index Encoding	7
3.2) TF-IDF.....	7
4. Task 3: Naive Bayes Implementations.....	8
4.1) Multinomial Naive Bayes.....	8
4.2) Bernoulli Naïve Bayes.....	8
5. Task 4: Numeric Meta-Features and Imputation.....	10
5.1) Injecting Missing Values	10
5.2) Handling Missing Values	10
6. Task 5: Outlier Detection and Treatment.....	11
6.1) IQR Detection.....	11
6.2) Clamping	11
7. Task 6: Scaling Study.....	12
8. Task 7: Class Imbalance and Performance Evaluation	13
8.1) Measuring Imbalance	13
8.2) Random Oversampling.....	13
8.3) Threshold Tuning	13
9. Task 8: Error Analysis.....	15
9.1) Eight Misclassified Headlines	15
10. Final Results Summary	17

LIST OF FIGURES

Figure 1 Confusion matrix at best threshold (validation set)	14
--	----

LIST OF TABLES

Table 1 Top 10 Tokens by DF	7
Table 2 Multinomial Naive Bayes Results Across Alpha Values.....	8
Table 3 Bernoulli Naive Bayes Results Across Alpha Values.....	8
Table 4 Mean vs Class-Conditional Imputation Results.....	10
Table 5 Validation Performance Before and After Clamping.....	11
Table 6 MinMax vs Standardisation Validation Results.....	12
Table 7 Threshold Tuning Results on Validation Set	13
Table 8 Best Threshold Performance Summary	13
Table 9 Eight Misclassified Headlines from the Test Set	15
Table 10 Model Performance Comparison Across Tasks.....	17

1. Dataset Overview and Individualisation

For this assignment I used the News Headlines Dataset for Sarcasm Detection from Kaggle, which has 28,619 headlines each labelled sarcastic (1) or not sarcastic (0). Each entry also includes an article URL.

My seed was calculated from my student ID: $277239 \bmod 100000 = 77239$. From this, $77239 \bmod 3 = 1$ gave me smoothing values $\alpha \in \{0, 0.5, 2\}$, $77239 \bmod 4 = 3$ gave ablation index $k = 3$, and $77239 \bmod 6 = 1$ so rare-word threshold $t = 1 + 2 = 3$. I shuffled the data with `random_state=77239` and split it 70/15/15, giving 20,033 training, 4,293 validation and 4,293 test rows.

2. Task 1: Tokenisation and Preprocessing

Building the preprocessing pipeline from scratch was a requirement, so I only used Python's `re` module and basic string operations throughout, without touching any external libraries like NLTK or spaCy.

2.1) Basic Cleaning

I lowercased each headline and removed HTML tags. Before stripping punctuation I replaced tonal markers with named tokens like `!!` becomes `[EXC_SEQ]`, `??` becomes `[QUE_SEQ]`, any mix of `?! or !?` becomes `[INT_MARK]`, and `...` becomes `[ELLIP]`. I did this first because if I had removed punctuation before this step these signals would have been lost completely. Apostrophes inside words were temporarily replaced with the placeholder `APOS` to protect contractions like `can't` and `we're`, then restored after punctuation removal. Tokenisation used the regex `[\w']+ | \[\w+\]` to pick up both regular words and the named marker tokens.

2.2) Negation Handling

I defined seven negation words: `not`, `no`, `never`, `don't`, `didn't`, `isn't`, `wasn't`. When one appears I count its vowels and prefix that many following tokens with `NOT_`. So `"not"` has one vowel and affects only the next token, while `"never"` has two vowels and reaches two tokens ahead. I preferred this over a fixed window since the vowel count at least ties the scope to something about the word itself rather than an arbitrary number.

2.3) Stopword Removal

I defined a custom stopwords list of 45 common function words. Removal happens after negation handling and any token already carrying a `NOT_` prefix is kept regardless, so negated tokens like `NOT_good` cannot be accidentally discarded.

2.4) N-gram Generation

On top of unigrams I also generated bigrams by pairing neighbouring tokens with an underscore, so `climate and change` become `climate_change`. Some sarcastic phrases only really make sense as a pair of words rather than individually, so including bigrams felt worthwhile.

2.5) OOV Handling

The vocabulary was built from training data only, giving 135,427 unique tokens. Any token in validation or test not found in this vocabulary is replaced with `<UNK>` to prevent unseen tokens from introducing noise.

3. Task 2: Manual Feature Representation

Both feature matrices were built entirely from scratch without CountVectorizer or TfidfVectorizer.

3.1) Count Vectoriser with Prime-Index Encoding

I computed document frequency for each token by counting how many training documents it appears in at least once. Tokens in fewer than 3 documents were dropped as too rare, leaving 11,797 tokens. These were sorted alphabetically and tokens at prime-numbered positions (2, 3, 5, 7, 11...) each got their own feature column, while tokens at non-prime positions were grouped into a single column at index 0. This gave 1,413 individual prime columns plus one non-prime bucket, totalling 1,414 features. The training matrix came out to shape (20033, 1414).

3.2) TF-IDF

Term frequency is the raw count of a token in a document. IDF was computed from the document frequencies above using:

$$\text{IDF}(t) = \log((1 + N) / (1 + \text{DF}(t))) + 1$$

The smoothing prevents division by zero. TF-IDF is the product of the two, then L2-normalised per row so longer headlines do not dominate.

Top 10 Tokens by Document Frequency

Rank	Token	DF
1	new	1144
2	trump	981
3	man	944
4	report	411
5	year	372
6	one	356
7	day	355
8	area	354
9	woman	348
10	says	346

Table 1: Top 10 Tokens by DF

The list is made up of general political and human-interest words that appear heavily across both sources, which highlights how much topical overlap there is between The Onion and HuffPost even though their tone is completely different.

4. Task 3: Naive Bayes Implementations

Both models run entirely in log-space throughout to avoid underflow when multiplying many small probabilities together.

4.1) Multinomial Naive Bayes

The log prior for each class c is $\log(\text{count}(c) / N_{\text{train}})$. The smoothed log-likelihood is:

$$\log P(\mathbf{t} | c) = \log((\text{count}(\mathbf{t}, c) + \alpha) / (\sum \text{count}(\mathbf{t}, c) + \alpha \cdot |V|))$$

When $\alpha = 0$, any zero count produces $\log(0) = -\infty$, so I replaced non-finite values with -10^9 to keep predictions stable. At prediction time the log posterior is the sum of the log prior and the dot product of the count vector with the log-likelihood vector. The class with the higher value is the prediction.

α	Val Acc	Val Macro F1	Test Acc	Test Macro F1
0	0.6003	0.5753	—	—
0.5	0.5998	0.5747	—	—
2	0.6007	0.5750	0.6203	0.5945

Table 2: Multinomial Naive Bayes Results Across Alpha Values

Both models perform almost identically, which makes sense given how short headlines are. Most tokens appear only once per headline anyway, so counting frequency versus just noting presence gives nearly the same result. Bernoulli edges slightly ahead on minority F1 (0.5952 vs 0.5945), likely because capping each token's contribution reduces sensitivity to noisy common words. For longer documents frequency would matter more, but here the difference is negligible.

The smoothing value makes almost no difference here, which makes sense given how large the vocabulary is and how rarely genuine zero-count events occur with 20,033 training documents. I selected $\alpha = 2$ as the best variant for the test evaluation.

4.2) Bernoulli Naïve Bayes

Bernoulli NB only looks at whether a token is present or absent, not how many times it appears. For each token and class I estimated:

$$P(\mathbf{t} \text{ present} | c) = (\text{count_present} + \alpha) / (nc + 2\alpha)$$

At inference time each feature contributes either the log-probability of presence or absence, and these are summed with the log prior. The 2α denominator accounts for the two possible Bernoulli outcomes.

α	Val Acc	Val Macro F1	Test Acc	Test Macro F1
0	0.6010	0.5776	—	—
0.5	0.6007	0.5774	—	—
2	0.6005	0.5758	0.6198	0.5952

Table 3: Bernoulli Naive Bayes Results Across Alpha Values

The two models perform almost identically, with less than 0.001 difference in accuracy. This is expected for short headlines since most tokens appear at most once per document anyway, so counting versus binary presence gives nearly the same information. Bernoulli NB edges ahead very slightly on minority-class F1 (0.5952 vs 0.5945 on test), probably because binary features cap any single token's contribution and are therefore a little less sensitive to noisy high-frequency terms.

5. Task 4: Numeric Meta-Features and Imputation

I extracted eleven numeric features from each raw headline and its URL: word count, character length, average word length, exclamation mark count, question mark count, negation word count, digit count, uppercase word count, sarcasm_ratio (uppercase_count divided by word_count, with 0.5 used when word_count is zero as the Ambiguity Constant), URL character length, and URL path depth. The domain name was excluded from all URL features to avoid leaking the class label. The training matrix shape was (20033, 11).

5.1) Injecting Missing Values

Using seed 77239, I randomly set 5% of all numeric training values to NaN, which gave 11,018 missing values in total including at least 1,008 in the sarcasm_ratio column as required by the assignment.

5.2) Handling Missing Values

I implemented two imputation strategies. Mean imputation replaces each missing value with the column mean across all non-missing training values for that feature. Class-conditional mean imputation computes a separate mean per class and uses the appropriate one for each missing entry.

Imputation Method	Val Accuracy	Val Macro F1
Mean imputation	0.6543	0.6235
Class-conditional mean	0.6553	0.6249

Table 4: Mean vs Class-Conditional Imputation Results

Class-conditional imputation performs better, and the reason is fairly intuitive. Sarcastic and non-sarcastic headlines have genuinely different distributions for features like uppercase_count and sarcasm_ratio. Replacing missing values with the overall column mean blends these two distributions together and introduces a small systematic bias. The class-conditional mean keeps within-class structure intact, which is exactly what Gaussian NB needs to estimate likelihoods accurately.

Mean imputation just fills in missing values using one overall average, so it mixes sarcastic and non-sarcastic examples together. That slightly washes out the differences between the two classes. If the mean is calculated separately for each class instead, those differences are kept, which is more in line with what Gaussian Naïve Bayes expects. The improvement is very small, only about 0.001 in accuracy and 0.0014 in macro F1, but it still makes more sense to use the class based approach when the feature distributions are not the same.

6. Task 5: Outlier Detection and Treatment

Some headlines are extreme in length, punctuation density, or digit count. Left untreated, these inflate Gaussian NB's within-class variance estimates and make the likelihood surface too broad, which weakens discrimination between classes.

6.1) IQR Detection

For each of the eleven features I computed Q1, Q3, and $IQR = Q3 - Q1$ on the class-conditionally imputed training data. Boundaries were set at $Q1 - 1.5 \times IQR$ and $Q3 + 1.5 \times IQR$. A row was flagged as an outlier if any of its feature values fell outside its feature boundary. In total 9,255 of the 20,033 training rows were flagged, which is high but typical for text-length features with long right tails.

6.2) Clamping

I chose to clamp rather than remove rows because deleting 9,255 rows would discard nearly half the training data. Clamping sets any value below the lower boundary to the boundary and any value above the upper boundary to the boundary. The class distribution stays unchanged at 10,530 non-sarcastic and 9,503 sarcastic since no rows are removed.

State	Val Accuracy	Val Macro F1
Before clamping	0.6553	0.6249
After clamping	0.6599	0.6554

Table 5: Validation Performance Before and After Clamping

Since rows were clamped rather than removed, the class distribution remains unchanged at 10,530 non-sarcastic and 9,503 sarcastic training examples.

This was the single largest improvement across all numeric-feature experiments, lifting macro F1 from 0.6249 to 0.6554. The extreme values were clearly distorting the variance estimates before clamping, and once they were contained the model's decision boundary sharpened noticeably.

7. Task 6: Scaling Study

Gaussian NB is theoretically unaffected by linear rescaling since it re-estimates class-conditional statistics from the training data. I tested this empirically by applying MinMax scaling and standardisation and comparing the results.

MinMax scaling: $x_{\text{scaled}} = (x - \min) / (\max - \min)$, mapped to $[0,1]$. Standardisation: $x_{\text{scaled}} = (x - \mu) / \sigma$, zero mean and unit variance. Both scalers were fit on clamped training data only.

Scaling Method	Val Accuracy	Val Macro F1
MinMax	0.6599	0.6554
Standardisation	0.6599	0.6554

Table 6: MinMax vs Standardisation Validation Results

Both methods gave identical results, confirming the theory. This makes sense when you think about it. If you scale all the inputs linearly, the model just learns scaled versions of the same parameters and ends up drawing the exact same boundary. So, scaling did not actually make the model any better here, it just keeps the numbers cleaner during computation.

8. Task 7: Class Imbalance and Performance Evaluation

8.1) Measuring Imbalance

The training set has 10,530 non-sarcastic (52.6%) and 9,503 sarcastic (47.4%) examples, giving an imbalance ratio of 1.108. This is mild but still enough to push the model towards the majority class, which shows up as many more false negatives than false positives in the confusion matrix.

8.2) Random Oversampling

I applied random oversampling to the minority class, drawing replacement samples from sarcastic training examples until both classes had 10,530 instances. Seed 77239 was used. The balanced Multinomial NB was then retrained with $\alpha = 0.5$.

8.3) Threshold Tuning

I computed softmax-normalised posterior probabilities for class 1 on the validation set and tested five threshold values:

Threshold	Accuracy	Macro F1	Minority F1
0.3	0.5556	0.4878	0.6741
0.4	0.5791	0.5422	0.6721
0.5	0.5989	0.5778	0.4835
0.6	0.5947	0.5478	0.4021
0.7	0.5802	0.5078	0.3190

Table 7: Threshold Tuning Results on Validation Set

Threshold 0.5 gave the highest macro F1 and was selected. Lower thresholds push the model to predict sarcasm more aggressively, which helps minority recall but creates many false positives and hurts overall macro F1. Higher thresholds do the opposite.

Metric	Value
Accuracy	0.5989
Precision (macro)	0.6141
Recall (macro)	0.5938
F1 macro	0.5778
F1 minority	0.4835

Table 8: Best Threshold Performance Summary

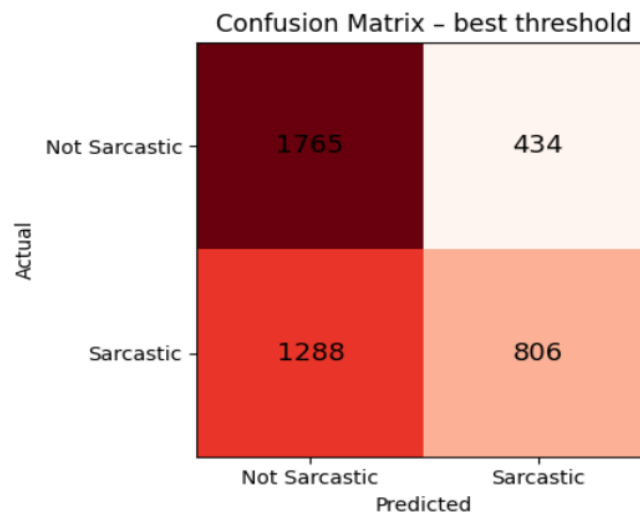


Figure 1: Confusion matrix at best threshold (validation set)

The confusion matrix makes the imbalance effect visible like 1,288 sarcastic headlines were called non-sarcastic, but only 434 went the other way. Even with oversampling the model is still more cautious about predicting sarcasm. Accuracy is misleading here since a dummy classifier that always predicts non-sarcastic would still hit around 52%, so macro F1 is the more meaningful metric. The low minority F1 of 0.4835 reflects just how hard sarcasm detection is at the word level, many sarcastic headlines use completely ordinary vocabulary and only become obviously satirical when you understand the context.

The dataset is only slightly imbalanced (52.6% non-sarcastic vs 47.4% sarcastic), but even this small gap pushes the model toward predicting the majority class more often. This shows up in the confusion matrix, where false negatives (1,288) are much higher than false positives (434). Because of this, accuracy can be misleading, as simply predicting the majority class would already give around 52%. Oversampling helps reduce this bias, but the minority F1 score (0.4835) is still relatively low, suggesting that sarcasm remains difficult to capture using only word-level features.

9. Task 8: Error Analysis

The final Multinomial NB model with $\alpha = 2$ was applied to the test set. Out of 4,293 test examples, 1,630 were misclassified, a test error rate of around 38%.

9.1) Eight Misclassified Headlines

S.NO	Headline	True	Pred	Error
1	rain-drenched cat announces it ready to stay inside and be part of family	Sarcastic	Not Sarcastic	FN
2	mother provides adult son with list of questions to ask doctor	Sarcastic	Not Sarcastic	FN
3	woman builds ironclad case proving mila kunis looks bad without makeup	Sarcastic	Not Sarcastic	FN
4	family assistants are the new nannies	Sarcastic	Not Sarcastic	FN
5	American students rank near bottom in math	Not Sarcastic	Sarcastic	FP
6	democrats see path to flipping state legislatures in 2018	Not Sarcastic	Sarcastic	FP
7	5 incredible female athletes who were also moms	Not Sarcastic	Sarcastic	FP
8	city council looks to hire diversity consultant	Not Sarcastic	Sarcastic	FP

Table 9: Eight Misclassified Headlines from the Test Set

The false negatives (1–4) are cases where the sarcasm is entirely compositional. A cat announcing it is ready to become part of the family is obviously absurd, but the individual tokens like cat, rain, family, announces are all perfectly ordinary news words. The model has no way to detect the incongruity between the subject and the formal register of the statement. Headline 2 reads exactly like a real lifestyle article at the word level. The sarcasm only becomes apparent when you understand that the scenario itself is being mocked.

The false positives (5–8) are real news stories that happen to use vocabulary the model has learned to associate with sarcasm. Words like 'incredible', 'flipping', and 'diversity consultant' appear frequently in Onion headlines where exaggerated political and social commentary is the style. The model conflates vocabulary that is common in satire with vocabulary that is common in serious political commentary, because the two genuinely overlap at the surface level.

Custom Headline Testing

- "Researchers publish study on climate change effects" (expected: not sarcastic) → CORRECT
- "Man in Kentucky decries influence of state in town affairs" (expected: not sarcastic) → CORRECT
- "Local restaurant in Cork wins award for best mediocre food" (expected: sarcastic) → WRONG

The first two headlines use flat neutral vocabulary the model correctly associates with real news. The third is built around the contradiction of 'best mediocre food', where a superlative is applied to a negative adjective. The model misses it because this bigram almost certainly never appears in training, and the individual words best and mediocre each appear in both classes without a strong signal either way. This is a good example of why bag-of-words fails at irony detection, the contradiction requires understanding that best and mediocre are semantically opposed, which word-count models simply cannot do.

10. Final Results Summary

Task	Model / Method	Val Acc	Val Macro F1
Task 3	Multinomial NB ($\alpha=2$, Count)	0.6007	0.5750
Task 3	Bernoulli NB ($\alpha=2$, Count)	0.6005	0.5758
Task 3 – Test	Multinomial NB	0.6203	0.5945
Task 3 – Test	Bernoulli NB	0.6198	0.5952
Task 4	Gaussian NB (mean imputation)	0.6543	0.6235
Task 4	Gaussian NB (class-cond. mean)	0.6553	0.6249
Task 5	Gaussian NB (after clamping)	0.6599	0.6554
Task 6	Gaussian NB (MinMax / Z-score)	0.6599	0.6554
Task 7	Multinomial NB (oversampled)	0.5989	0.5778

Table 10: Model Performance Comparison Across Tasks

Gaussian NB on numeric features consistently outperformed the text-based models by around 5–6 macro F1 points on the validation set, which suggests that surface-level properties like headline length and capitalisation are more discriminative than individual word identities for this task. The biggest single improvement came from outlier clamping, which added 0.03 to macro F1 by stopping extreme values from inflating variance estimates. Both scaling methods gave identical results as expected. The text-based models were almost insensitive to the smoothing parameter at this training set size. The oversampled model's lower numbers compared to the numeric-feature models reflect the fundamental difficulty of detecting sarcasm from vocabulary alone when the linguistic signal is semantic and compositional rather than lexical.